

MI HDMI API

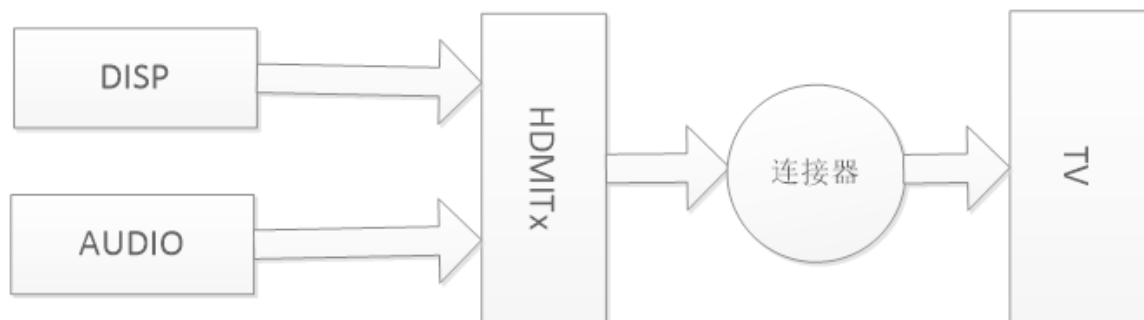
1 Overview

1.1. Module Description

HDMI (High Definition Multimedia Interface), namely high-definition multimedia interface, is a fully digital video and sound transmission interface, which can transmit uncompressed audio and video signals at the same time.

HDMI can be divided into **TX 端** and **RX端**, TX is the source device, used to transmit data to RX (sink device). This module currently only supports HDMITx based on HDMI ver1.4.

1.2. Data flow block diagram



Picture 1-1

1.3. Features of each platform

Table 1-1

Platform Support Features	Taiyaki/Takoyaki
1920x1080@24	Y
1920x1080@25	Y
1920x1080@30	Y
1920x1080@50	Y
1920x1080@60	Y
1280x720@60	Y
1280x720@50	Y
720x576@50	Y
720x480@60	Y
1280x1024@60	Y
1024x768@60	Y
1440x900@60	Y
1600x1200@60	Y
1280x800@60	Y
1366x768@60	Y
1680x1050@60	Y
Platform 3840x2160@30 Support Features	Taiyaki/Takoyaki Y

2560x1440@30	Y
Audio_32K	Y
Audio_48K	Y
1920x1080@59.94	N
1920x1080@29.97	N
1280x720@59.94	N
1280x720@29.97	N

1.4. Keyword Description

- EDID

Extended display identification data

Refers to the screen resolution information, including the manufacturer's name and serial number.

- InfoFrame

The HDMI specification, which belongs to the auxiliary data category, is used to inform the sink device of various characteristics of the data to be transmitted.

- DeepColor

That is, the dark mode is used to represent more accurate colors, and the amount of data will increase accordingly.

- SinkInfo

Device information on the sink side.

2. API Reference

2.1. Function Module API

table 2-1

API name	Features
MI_HDMI_Init [#22-mi_hdmi_init]	Initialize HDMI
MI_HDMI_DeInit [#23-mi_hdmi_deinit]	deinitialize HDMI
MI_HDMI_Open [#24-mi_hdmi_open]	Open HDMI
MI_HDMI_Close [#25-mi_hdmi_close]	Turn off HDMI
MI_HDMI_SetAttr [#26-mi_hdmi_setattr]	Set HDMI properties
MI_HDMI_GetAttr [#27-mi_hdmi_getattr]	Get HDMI properties
MI_HDMI_Start [#28-mi_hdmi_start]	enable HDMI
MI_HDMI_Stop [#29-mi_hdmi_stop]	stop HDMI
MI_HDMI_GetSinkInfo [#210-mi_hdmi_getsinkinfo]	Get the capability information of HDMI sink
MI_HDMI_SetAvMute [#211-mi_hdmi_setavmute]	Set HDMI audio and video mute function
MI_HDMI_ForceGetEdid [#212-mi_hdmi_forcegetedid]	Get HDMI EDID information
MI_HDMI_SetDeepColor [#213-mi_hdmi_setdeepcolor]	Set HDMI Deep Color Mode
MI_HDMI_GetDeepColor [#214-mi_hdmi_getdeepcolor]	Get HDMI Deep Color Mode
MI_HDMI_SetInfoFrame [#215-mi_hdmi_setinfoframe]	Set HDMI Info Frame information
API name	Features

MI_HDMI_GetInfoFrame [#216-mi_hdmi_getinfoframe]	Get the Info Frame information of the currently set HDMI
MI_HDMI_SetAnalogDrvCurrent [#217-mi_hdmi_setanalogdrvcurrent]	Set HDMI channel drive current
MI_HDMI_InitDev [#218-mi_hdmi_initdev]	Initialize HDMI device
MI_HDMI_DeInitDev [#219-mi_hdmi_deinitdev]	Deinitialize HDMI device

.....

2.2. MI_HDMI_Init

- Features

Initialize the HDMI module.

- grammar

```
MI_S32 MI_HDMI_Init(MI_HDMI_InitPara_t *pstHdmiPara);
```

- formal parameter

Table 2-2

parameter name	describe	input Output
pstHdmiPara	Initialize the parameter structure pointer, which can be configured as NULL	enter

- return value

0 Initialization succeeded.

Non-zero initialization failed, please refer to the [error code](#) [#jump] for details .

- rely

- Header files: mi_hdmi.h, mi_hdmi_datatype.h

- Library file: libmi_hdmi.a
- Notice

After HDMI registers the event callback, the system will notify the AP of hot swapping and other operations in the form of callback. If the user does not need to make a special response to the corresponding event, the function pointer in the initialization parameter structure can be set to NULL.

- Special Instructions

The pfnHdmiEventCallback callback registered in Init is mainly to call back the plug and unplug events to the App. The current design is to open a thread at the SDK layer, and then check the corresponding time and report it to the callback registrant

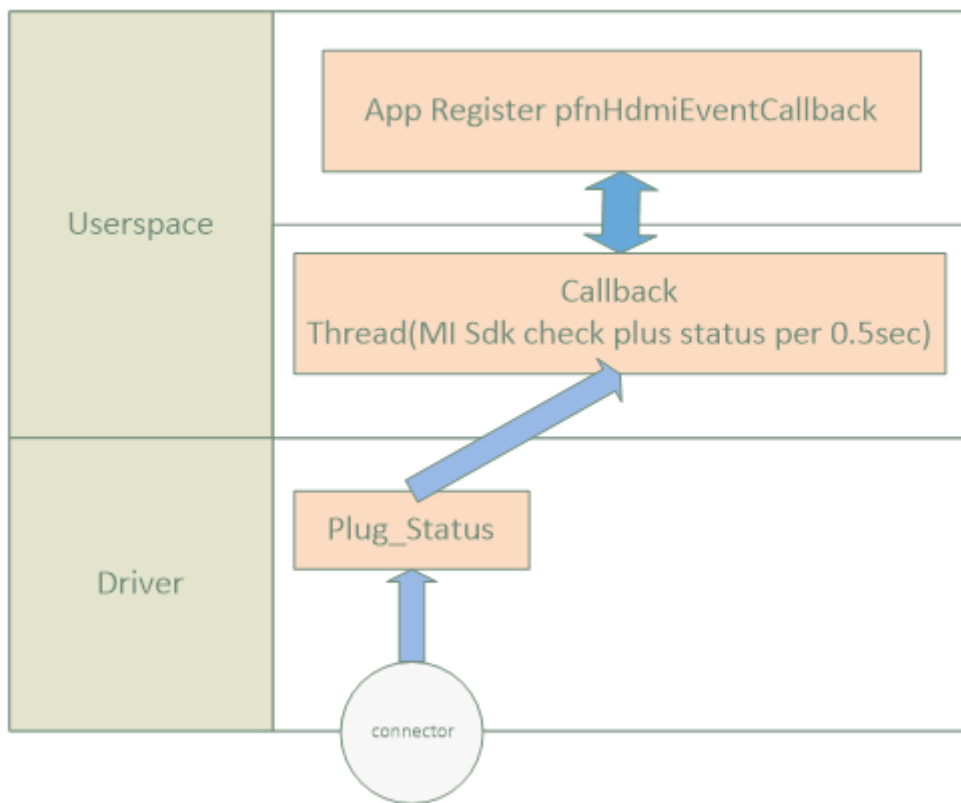


diagram 2-1

- Example

```
1. MI_HDMI_InitPara_t stHdmiPara ; _
2. MI_HDMI_Attr_t stAttr ; _
3. MI_HDMI_TimingType_eTimingType ; _ _
```

```
4. MI_HDMI_Edid_t stEdid ; _
5. /    * Start DISP */
6. eTimingType = E_MI_HDMI_TIMING_1080_60P;
7. /* 启动HDMI */
8. stHdmiPara.pCallbackArgs = NULL;
9. stHdmiPara.pfnHdmiEventCallback = NULL;
10. MI_HDMI_Init(&stHdmiPara);
11. MI_HDMI_Open(0);
12. /* 获取sink设备支持的timing */
13. MI_HDMI_ForceGetEdid(0, &stEdid);
14. ...
15. /* 根据sink 支持的timing进行配置 */
16. MI_MPI_HDMI_GetAttr(0, &stAttr);
17. stAttr.bEnableHdmi = MI_TRUE;
18. stAttr.bEnableVideo = MI_TRUE;
19. stAttr.eVideoFmt = enVideoFmt;
20. stAttr.eVidOutMode= E_MI_HDMI_VIDEO_MODE_YCBCR444;
21. stAttr.eDeepColorMode = E_MI_HDMI_DEEP_COLOR_24BIT;
22. stAttr.bEnableAudio = MI_FALSE;
23. stAttr.bIsMultiChannel = MI_FALSE;
24. stAttr.eBitDepth = E_MI_HDMI_BIT_DEPTH_16;
25. MI_HDMI_SetAttr(0, &stAttr);
26. MI_HDMI_Start(0);
27. /* 停止HDMI */
```

```
28. MI_HDMI_Stop(0);
```

```
29. MI_HDMI_Close(0);
```

```
30. MI_HDMI_DeInit();
```

- 相关主题

[MI_HDMI_DeInit](#) [#23-mi_hdmi_deinit]

2.3. MI_HDMI_DeInit

- 功能

去初始化 HDMI。

- 语法

```
MI_S32 MI_HDMI_DeInit();
```

- 返回值

- 0 去初始化成功。
- 非0 去初始化失败，详情参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_hdmi.h、mi_hdmi_datatype.h
- 库文件：libmi_hdmi.a

- 举例

请参见 [MI_HDMI_Init](#) [#22-mi_hdmi_init] 的举例。

- 相关主题

[MI_HDMI_Init](#) [#22-mi_hdmi_init]

2.4. MI_HDMI_Open

- 功能

打开 HDMI 。

- 语法

```
MI_S32 MI_HDMI_Open(MI_HDMI_DeviceId_e eHdmi);
```

- 形参

表2-3

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入

- 返回值
 - 0 打开HDMI设备成功。
 - 非0打开设备失败，详情参照[错误码](#) [#jump]。

- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a

- 注意

打开 HDMI 之前必须先调用[MI_HDMI_Init](#) [#22-mi_hdmi_init]

重复打开 HDMI 返回成功。

- 举例

请参见 [MI_HDMI_Init](#) [#22-mi_hdmi_init] 的举例。
- 相关主题

[MI_HDMI_Close](#) [#25-mi_hdmi_close]

2.5. MI_HDMI_Close

- 功能

关闭 HDMI。

- 语法

```
MI_S32 MI_HDMI_Close(MI_HDMI_DeviceId_e eHdmi);
```

- 形参

表2-4

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入

- 返回值

- 0 销毁HDMI设备成功。
- 非0销毁HDMI设备失败，详情参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_hdmi.h、mi_hdmi_datatype.h
- 库文件：libmi_hdmi.a

- 注意

重复关闭返回成功。

- 举例

请参见 [MI_HDMI_Init](#) [#22-mi_hdmi_init]的举例。

- 相关主题

[MI_HDMI_Open](#) [#24-mi_hdmi_open]

.....

2.6. MI_HDMI_SetAttr

- 功能

设置 HDMI 属性。

• 语法

```
MI_S32 MI_HDMI_SetAttr(MI_HDMI_DeviceId_e
eHdmi,MI_HDMI_Attr_t*pstAttr);
```

• 形参

表2-5

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入
pstAttr	HDMI 属性结构体指针。	输入

• 返回值

- 0 设置HDMI设备属性成功。
- 非0设置HDMI设备属性失败，详情参照[错误码](#) [#jump]。

• 依赖

- 头文件：mi_hdmi.h、mi_hdmi_datatype.h
- 库文件：libmi_hdmi.a

• 注意

- 设置 HDMI 属性之前需要调用[MI_HDMI_Open](#) [#24-mi_hdmi_open]。
- 用户需要在调用[MI_HDMI_Start](#) [#28-mi_hdmi_start] 之前设置 HDMI 属性。
- 在调用 [MI_HDMI_Start](#) [#28-mi_hdmi_start] 之后设置 HDMI 属性，需要先调用[MI_HDMI_Stop](#) [#29-mi_hdmi_stop] 再设置新的 HDMI 属性。
- 传输 audio时需要确保有传输视频数据，音频是夹到视频中传输的

• 举例

```
1. MI_HDMI_InitPara_t stHdmiPara;

2. MI_HDMI_Attr_t  stAttr;
```

```
3. MI_HDMI_TimingType_e enVideoFmt;

4. /* 启动DISP */

5. /* HDMI start后设置HDMI属性 */

6. MI_HDMI_Stop(0, MI_TRUE);

7. MI_HDMI_GetAttr(0, &stAttr);

8. /* 用户更改HDMI属性，如timing */

9. stAttr.eVideoFmt = E_MI_HDMI_VIDEO_FMT_720P_60;

10. MI_HDMI_SetAttr(0, &stAttr);

11. MI_HDMI_Start(0, MI_FALSE);
```

- 相关主题

[MI_HDMI_GetAttr](#) [#27-mi_hdmi_getattr]

2.7. MI_HDMI_GetAttr

- 功能

获取 HDMI 属性。

- 语法

```
MI_S32 MI_HDMI_GetAttr(MI_HDMI_DeviceId_e eHdmi,
MI_HDMI_Attr_t *pstAttr);
```

- 语法

表2-6

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入
pstAttr	HDMI 属性结构体指针。	输出

- 返回值
 - 0 获取HDMI设备属性成功。
 - 非0获取HDMI设备属性失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 举例

请参见 [MI_HDMI_Init](#) [#22-mi_hdmi_init]举例。
- 相关主题

[MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]

2.8. MI_HDMI_Start

- 功能

启动 HDMI。
- 语法

```
MI_S32 MI_HDMI_Start(MI_HDMI_DeviceId_e eHdmi);
```

- 形参

表2-7

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入

- 返回值
 - 0 启动HDMI工作成功。
 - 非0 HDMI设备启动失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 注意
 - 启动 HDMI 前必须先调用 [MI_HDMI_Open](#) [#24-mi_hdmi_open]
- 举例

请参见 [MI_HDMI_Init](#) [#22-mi_hdmi_init]举例。
- 相关主题

[MI_HDMI_Stop](#) [#29-mi_hdmi_stop]

2.9. MI_HDMI_Stop

- 功能

停止 HDMI设备工作。
- 语法

```
MI_S32 MI_HDMI_Stop(MI_HDMI_DeviceId_e eHdmi);
```

- 形参
- 表2-8

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入

- 返回值
 - 0停止HDMI设备工作。
 - 非0 停止HDMI设备失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 相关主题

[MI_HDMI_Start](#) [#28-mi_hdmi_start]

2.10. MI_HDMI_GetSinkInfo

- 功能

获取 HDMI Sink 端的能力级信息。
- 语法

```
MI_S32 MI_HDMI_GetSinkInfo(MI_HDMI_DeviceId_e eHdmi,  
MI_HDMI_SinkInfo_t *pstSinkInfo);
```

- 形参

表2-9

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。取值范围：0	输入
pstSinkCap	HDMI Sink 端能力级结构体指针	输出

- 返回值
 - 0 获取HDMI Sink端能力成功。
 - 非0获取Sink信息失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 注意
 - 调用前必须先打开 HDMI。为了确保 HDMI 打开完成，建议在调用 [MI_HDMI_Open](#) [#24-mi_hdmi_open] 接口后等待 500ms 左右。
 - 应在 HDMI 启动且插入线缆之后调用。
 - 一般在插入事件处理函数中或者强制获取 EDID 之后使用。
- 举例

```
1. MI_HDMI_Init_Para_t stHdmiPara;  
2. MI_HDMI_Attr_t stAttr;  
3. MI_HDMI_TimingType_e enVideoFmt;  
4. MI_HDMI_SinkInfo_t stSinkInfo;  
5. /* 启动DISP */  
6. /* 启动HDMI */  
7. /* 获取HDMI Sink端能力级 */  
8. MI_HDMI_GetSinkInfo(0, &stSinkInfo);
```

2.11. MI_HDMI_SetAvMute

- 功能

设置 HDMI 音视频静音功能。
- 语法

MI_S32 MI_HDMI_SetAvMute(MI_HDMI_DeviceId_e eHdmi, MI_BOOL bAvMute);

• 形参

表2-10

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入
bAvMute	是否设置 HDMI 音视频静音。取值范围： MI_TRUE：设置音视频静音。 MI_FALSE：关闭音视频静音。	输入

• 返回值

- 0 成功。
- 非0失败，详情参照[错误码](#) [#jump]。

• 依赖

- 头文件：mi_hdmi.h、mi_hdmi_datatype.h
- 库文件：libmi_hdmi.a

• 注意

需要在调用[MI_HDMI_Start](#) [#28-mi_hdmi_start]之后使用。

• 举例

```
1. MI_HDMI_Start(0);  
2. MI_HDMI_SetMute(0, MI_TRUE);
```

2.12. MI_HDMI_ForceGetEdid

• 功能

获取 HDMI 的 EDID 信息。

• 语法

```
MI_S32 MI_HDMI_ForceGetEdid(MI_HDMI_DeviceId_e
eHdmi,MI_HDMI_Edid_t *pstEdidData);
```

• 形参

表2-11

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入
pstEdidData	HDMI 的 EDID 信息。	输出

• 返回值

- 0 成功。
- 非0 失败，详情参照[错误码](#) [#jump]。

• 依赖

- 头文件：mi_hdmi.h、mi_hdmi_datatype.h
- 库文件：libmi_hdmi.a

• 注意

- 调用前必须先打开 HDMI。

• 举例

```
1. MI_HDMI_InitPara_t stHdmiPara;

2. MI_HDMI_Attr_t stAttr;

3. MI_HDMI_Edid_t stEdidData;

4. /* 启动DISP */

5. /* 启动HDMI */

6. /* 获取HDMI EDID信息 */
```

```
7. MI_HDMI_ForceGetEdid(0, &stEdidData);
```

2.13. MI_HDMI_SetDeepColor

- 功能

设置 Deep Color 模式。

- 语法

```
MI_S32 MI_HDMI_SetDeepColor(MI_HDMI_DeviceId_e  
eHdmi,MI_HDMI_DeepColor_e eDeepColor);
```

- 形参

表2-12

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	
输入		
eDeepColor	HDMI 的 Deep Color 模式。	输入

- 返回值

-0成功。

- 非0失败，详情参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_hdmi.h、mi_hdmi_datatype.h
- 库文件：libmi_hdmi.a

- 注意

- 该接口属于高级接口，一般不需要调用。

- 调用前必须先打开 HDMI。
- 该接口可用于设置 [MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]不支持的某些 Deep Color 模式。
- 该接口暂未实现。
- 举例

```
1. MI_HDMI_InitPara_t stHdmiPara;  
2. MI_HDMI_Attr_t stAttr;  
3. /* 启动 DISP*/  
4. /* 启动 HDMI */  
5. /* 设置 Deep color 模式 */  
6. MI_HDMI_SetDeepColor (0, E_MI_HDMI_DEEP_COLOR_24BIT);
```

2.14. MI_HDMI_GetDeepColor

- 功能
获取 Deep Color 模式。
- 语法

```
MI_S32 MI_HDMI_GetDeepColor(MI_HDMI_DeviceId_e eHdmi,  
MI_HDMI_DeepColor_e *peDeepColor);
```

- 形参

表2-13

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。取值范围：0	输入
peDeepColor	HDMI 的 Deep Color 模式指针。	输入

- 返回值
 - 0 成功。
 - 非0初始化失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 注意
 - 该接口属于高级接口，一般不需要调用。
 - 调用前必须先打开 HDMI。
 - 该接口暂未实现。
- 相关主题

[MI_HDMI_SetDeepColor](#) [#213-mi_hdmi_setdeepcolor]

2.15. MI_HDMI_SetInfoFrame

- 功能
设置 HDMI 的信息帧。
- 语法

```
MI_S32 MI_HDMI_SetInfoFrame(MI_HDMI_DeviceId_e eHdmi, MI_HDMI_InfoFrame_t *pstInfoFrame);
```

- 形参

表2-14

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入
pstInfoFrame	图像信息帧结构体指针。	输入

- 返回值
 - 0成功。
 - 非0失败，详情参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 注意
 - 该接口一般用于用户态回调函数中配置信息帧属性。其他情况，一般不需调用本接口。
 - 调用前必须先打开 HDMI。
 - 本接口只支持设置 **AVI**、**SPD** 和 **AUDIO** 信息帧，其他类型的信息帧设置无效。
- 举例

```
1. MI_HDMI_DeviceId_e eHdmi = E_MI_HDMI_ID_0;

2.
MI_HDMI_InfoFrame_t stAviInfoFrame = {E_MI_HDMI_INFOFRAME_TYPE_AV

3.
MI_HDMI_InfoFrame_t stAudInfoFrame = {E_MI_HDMI_INFOFRAME_TYPE_AU

4.
MI_HDMI_InfoFrame_t stSpdInfoFrame = {E_MI_HDMI_INFOFRAME_TYPE_SP

5. /* AviInfoFrame */

6.
stAviInfoFrame.unInforUnit.stAviInfoFrame.bEnableAfdOverWrite = 0

7. ...

8. /* AudioInfoFrame */

9.
```

```
stAudInfoFrame.unInforUnit.stAudInfoFrame.u32ChannelCount = 2;

10. ...

11. /* SpdInfoFrame */

12.
stSpdInfoFrame.unInforUnit.stSpdInfoFrame.au8VendorName[0] = 0;

13. ...

14. MI_HDMI_SetInfoFrame(eHdmi, &stAviInfoFrame);
```

- 相关主题

[MI_HDMI_GetInfoFrame](#) [#216-mi_hdmi_getinfoframe]

2.16. MI_HDMI_GetInfoFrame

- 功能

获取 HDMI 的信息帧。

- 语法

```
MI_RESULT MI_HDMI_GetInfoFrame(MI_HDMI_DeviceId_e
eHdmi,MI_HDMI_InfoFrameType_e
eInfoFrameType,MI_HDMI_InfoFrame_t *pstInfoFrame);
```

- 形参

表2-15

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入
eInfoFrameType	信息帧类型。	输入
pstInfoFrame	图像信息帧结构体指针。	输入

- 返回值
 - 0 初始化成功。
 - 非0 初始化失败，详情参照[错误码 \[#jump\]](#)。
- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 注意

该接口一般用于用户态回调函数中获取信息帧属性。其他情况，一般不需调用本接口。

2.17. MI_HDMI_SetAnalogDrvCurrent

- 功能
设置 HDMI 通道的驱动电流
- 语法

```
MI_RESULT MI_HDMI_SetAnalogDrvCurrent(MI_HDMI_DeviceId_e eHdmi, MI_HDMI_AnalogDrvCurrent_t *pstAnalogDrvCurrent);
```

- 形参

表2-16

参数名称	描述	输入/输出
eHdmi	HDMI 接口号。 取值范围：0	输入
pstAnalogDrvCurrent	HDMI 输出电流的结构体类型	输入

- 返回值
 - 0 初始化成功。
 - 非0 初始化失败，详情参照[错误码 \[#jump\]](#)。

- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
- 注意
 - 该接口属于高级接口，提供给用户调整hdmi的通道输出电流。
 - 调用前必须先打开 HDMI。
- 举例

```
1. MI_HDMI_AnalogDrvCurrent_t CurInfo = {};  
2. memset(&CurInfo, 0xFF, sizeof(CurInfo));  
3. MI_HDMI_SetAnalogDrvCurrent(g_HdmiAttr.eHdmi, &CurInfo);
```

2.18. MI_HDMI_InitDev

- 功能
初始化hdmi设备
- 语法

```
MI_S32 MI_HDMI_InitDev(MI_HDMI_InitParam_t *pstInitParam);
```

- 形参

表2-17

参数名称	描述	输入/输出
pstInitParam	初始化参数结构体指针，可以配置为NULL	输入

- 返回值
 - 0 初始化成功。
 - 非0 初始化失败，详情参照错误码。

- 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
 - 注意
 - 此接口在Version 2.06以上版本推荐使用，用于替换原有MI_HDMI_Init接口。
-

2.19. MI_HDMI_DeInitDev

- 功能
反初始化HDMI设备
- 语法

```
MI_S32 MI_HDMI_DeInitDev(void);
```

- 返回值
 - 0 初始化成功。
 - 非0 初始化失败，详情参照错误码。
 - 依赖
 - 头文件：mi_hdmi.h、mi_hdmi_datatype.h
 - 库文件：libmi_hdmi.a
 - 注意
此接口在Version 2.06以上版本推荐使用，用于替换原有MI_HDMI_DeInit
[#23-mi_hdmi_deinit]接口。
-

3. HDMI 数据类型

3.1. 数据类型定义

表3-1

数据类型	定义
MI_HDMI_DeviceId_e [#32-mi_hdmi_deviceid_e]	定义 HDMI 接口号
MI_HDMI_InitPara_t [#33-mi_hdmi_initpara_t]	定义 HDMI 初始化属性结构体
MI_HDMI_EventCallBack [#34-mi_hdmi_eventcallback]	定义 HDMI 回调函数
MI_HDMI_EventType_e [#35-mi_hdmi_eventtype_e]	定义 HDMI 事件通知类型
MI_HDMI_Attr_t [#36-mi_hdmi_attr_t]	定义 HDMI 属性结构体
MI_HDMI_TimingType_e [#37-mi_hdmi_timingtype_e]	定义 HDMI 视频制式类型
MI_HDMI_OutputMode_e [#38-mi_hdmi_outputmode_e]	定义 HDMI 颜色空间类型
MI_HDMI_ColorType_e [#39-mi_hdmi_colortype_e]	定义 HDMI 输出颜色类型
MI_HDMI_DeepColor_e [#310-mi_hdmi_deepcolor_e]	定义 HDMI 深色模式类型
MI_HDMI_SampleRate_e [#311-mi_hdmi_samplerate_e]	定义 HDMI 音频输出采样率类型
MI_HDMI_BitDepth_e [#312-mi_hdmi_bitdepth_e]	定义 HDMI 音频输出采样位宽类型
MI_HDMI_SinkInfo_t [#313-mi_hdmi_sinkinfo_t]	定义 HDMI Sink 端能力级结构体
MI_HDMI_Edid_t [#314-MI_HDMI_Edid_t]	定义 HDMI EDID 信息结构体
MI_HDMI_InfoFrameType_e [#315-mi_hdmi_infotype_e]	定义 HDMI 信息帧类型
MI_HDMI_AviInfoFrameVer_t [#316-mi_hdmi_aviinfoframever_t]	定义 AVI 信息帧结构体
MI_HDMI_AudInfoFrameVer_t [#317-mi_hdmi_audinfoframever_t]	定义 AUDIO 信息帧结构体
数据类型	定义

MI_HDMI_SpdInfoFrame_t [#318-mi_hdmi_spdinfoframe_t]	定义 SPD 信息帧结构体
MI_HDMI_InfoFrame_Unit_u [#319-mi_hdmi_infoframe_unit_u]	定义 HDMI 信息帧联合体
MI_HDMI_AnalogDrvCurrent_t [#320-mi_hdmi_analogdrvcurrent_t]	定义 HDMI 通道驱动电流结构体

注：本节已涵盖各重要的数据类型，部分未列出数据类型请参见 [mi_hdmi_datatype.h](#)

3.2. MI_HDMI_DeviceId_e

- 说明
定义 HDMI 接口号。
- 定义

```
typedef enum
{
    E_MI_HDMI_ID_0 = 0,
    E_MI_HDMI_ID_BUTT
} MI_HDMI_DeviceId_e;
```

- 成员

表3-2

成员名称	描述
E_MI_HDMI_ID_0	HDMI 接口 0

3.3. MI_HDMI_InitPara_t

- 说明
定义 HDMI 初始化属性结构体。
- 定义

```
typedef struct MI_HDMI_InitPara_s
{
    MI_HDMI_EventCallback pfnHdmiEventCallback;

    void *pCallbackArgs;
}MI_HDMI_InitPara_t;
```

- 成员

表3-3

成员名称	描述
pfnHdmiEventCallback	事件处理回调函数
pCallbackArgs	回调函数参数私有数据

- 相关数据类型及接口
[MI_HDMI_Init](#) [#22-mi_hdmi_init]

3.4. MI_HDMI_EventCallBack

- 说明
定义 HDMI 回调函数。
- 语法

```
typedef MI_RESULT (*MI_HDMI_EventCallBack)
(MI_HDMI_DeviceId_e eHdmi, MI_HDMI_EventType_e event, void
```

```
entParam, void *pUsrParam);
```

- 成员

表3-4

成员名称	描述
MI_HDMI_EventType_e	HDMI事件类型
pEventParam pUsrParam	事件私有数据

3.5. MI_HDMI_EventType_e

- 说明

定义 HDMI 事件通知类型。

- 定义

```
typedef enum
{
    E_MI_HDMI_EVENT_HOTPLUG = 0,
    E_MI_HDMI_EVENT_NO_PLUG,
    E_MI_HDMI_EVENT_MAX
} MI_HDMI_EventType_e;
```

- 成员

表3-5

成员名称	描述
E_MI_HDMI_EVENT_HOTPLUG	HDMI Cable 连接事件
E_MI_HDMI_EVENT_NO_PLUG	HDMI Cable 断开连接事件

- 相关数据类型及接口

[MI_HDMI_Init](#) [#22-mi_hdmi_init]

3.6. MI_HDMI_Attr_t

- 说明

定义 HDMI 属性结构体。

- 定义

```
typedef struct MI_HDMI_Attr_s
{
    MI_HDMI_VideoAttr_t stVideoAttr;

    MI_HDMI_AudioAttr_t stAudioAttr;

    MI_HDMI_EnInfoFrame_t stEnInfoFrame;
} MI_HDMI_Attr_t;

typedef struct MI_HDMI_VideoAttr_s
{
    MI_BOOL bEnableVideo;

    MI_HDMI_TimingType_e eTimingType;

    MI_HDMI_OutputMode_e eOutputMode;

    MI_HDMI_ColorType_e eColorType;

    MI_HDMI_ColorType_e eInColorType;
```

```

        MI_HDMI_DeepColor_e eDeepColorMode;
    } MI_HDMI_VideoAttr_t;

typedef struct MI_HDMI_AudioAttr_s
{
    MI_BOOL bEnableAudio;

    MI_BOOL bIsMultiChannel;

    MI_HDMI_SampleRate_e eSampleRate;

    MI_HDMI_BitDepth_e eBitDepth;
} MI_HDMI_VideoAttr_t;

typedef struct MI_HDMI_EnInfoFrame_s
{
    MI_BOOL bEnableAviInfoFrame;

    MI_BOOL bEnableAudInfoFrame;

    MI_BOOL bEnableSpdInfoFrame;

    MI_BOOL bEnableMpegInfoFrame;
} MI_HDMI_EnInfoFrame_t;

```

- 成员

表3-6

成员名称	描述
bEnableVideo	是否输出视频
eTimingType	视频制式，此参数需要与视频输出配置的制式保持一致
eOutputMode	HDMI 输出视频模式
eColorType	HDMI 输出颜色类型
eInColorType	HDMI 输入颜色类型
eDeepColorMode	DeepColor 输出模式。 默认为 MI_HDMI_DEEP_COLOR_24BIT 不支持。 MI_HDMI_DEEP_COLOR_30BIT 和 MI_HDMI_DEEP_COLOR_36BIT。 MI_HDMI_DEEP_COLOR_48BIT
bEnableAudio	是否使能音频
blsMultiChannel	多声道数 0：2声道； 1：8 声道。/目前不支持
enSampleRate	音频采样率，此参数需要与 AO 的配置保持一致
enBitDepth	音频位宽，默认为 16，此参数需要与 AO 的配置保持一致
bEnableAviInfoFrame	是否使能 AVI InfoFrame 建议使能
bEnableAudInfoFrame	是否使能 AUDIO InfoFrame 建议使能
bEnableSpdInfoFrame	是否使能 SPD InfoFrame

- 注意事项
 - 用户可以根据建议值设置 HDMI 属性
- 相关数据类型及接口

3.7. MI_HDMI_TimingType_e

- 说明
定义 HDMI 视频制式类型。
- 定义

```
typedef enum
{
    E_MI_HDMI_TIMING_480_60I = 0,
    E_MI_HDMI_TIMING_480_60P = 1,
    E_MI_HDMI_TIMING_576_50I = 2,
    E_MI_HDMI_TIMING_576_50P = 3,
    E_MI_HDMI_TIMING_720_50P = 4,
    E_MI_HDMI_TIMING_720_60P = 5,
    E_MI_HDMI_TIMING_1080_50I = 6,
    E_MI_HDMI_TIMING_1080_50P = 7,
    E_MI_HDMI_TIMING_1080_60I = 8,
    E_MI_HDMI_TIMING_1080_60P = 9,
    E_MI_HDMI_TIMING_1080_30P = 10,
    E_MI_HDMI_TIMING_1080_25P = 11,
    E_MI_HDMI_TIMING_1080_24P = 12,
    E_MI_HDMI_TIMING_4K2K_30P = 13,
    E_MI_HDMI_TIMING_1440_50P = 14,
    E_MI_HDMI_TIMING_1440_60P = 15,
```

E_MI_HDMI_TIMING_1440_24P = 16,
E_MI_HDMI_TIMING_1440_30P = 17,
E_MI_HDMI_TIMING_1470_50P = 18,
E_MI_HDMI_TIMING_1470_60P = 19,
E_MI_HDMI_TIMING_1470_24P = 20,
E_MI_HDMI_TIMING_1470_30P = 21,
E_MI_HDMI_TIMING_1920x2205_24P = 22,
E_MI_HDMI_TIMING_1920x2205_30P = 23,
E_MI_HDMI_TIMING_4K2K_25P = 24,
E_MI_HDMI_TIMING_4K1K_60P = 25,
E_MI_HDMI_TIMING_4K2K_60P = 26,
E_MI_HDMI_TIMING_4K2K_24P = 27,
E_MI_HDMI_TIMING_4K2K_50P = 28,
E_MI_HDMI_TIMING_2205_24P = 29,
E_MI_HDMI_TIMING_4K1K_120P = 30,
E_MI_HDMI_TIMING_4096x2160_24P = 31,
E_MI_HDMI_TIMING_4096x2160_25P = 32,
E_MI_HDMI_TIMING_4096x2160_30P = 33,
E_MI_HDMI_TIMING_4096x2160_50P = 34,
E_MI_HDMI_TIMING_4096x2160_60P = 35,
E_MI_HDMI_TIMING_1024x768_60P = 36,
E_MI_HDMI_TIMING_1280x1024_60P = 37,
E_MI_HDMI_TIMING_1440x900_60P = 38,
E_MI_HDMI_TIMING_1600x1200_60P = 39,
E_MI_HDMI_TIMING_1280x800_60P = 40,

```

E_MI_HDMI_TIMING_1366x768_60P = 41,

E_MI_HDMI_TIMING_1680x1050_60P = 42,

E_MI_HDMI_TIMING_1920x1080_5994P = 43,

E_MI_HDMI_TIMING_1920x1080_2997P = 44,

E_MI_HDMI_TIMING_1280x720_5994P = 45,

E_MI_HDMI_TIMING_1280x720_2997P = 46,

E_MI_HDMI_TIMING_MAX,

} MI_HDMI_TimingType_e;

```

- 注意事项
 - 需要根据视频输出的制式设置相应的 HDMI 的制式。
 - 当AP在设置timing的时候，需要匹配DISP模块的timing，HDMI的timing需和DISP模块的timing保持一致。
- 相关数据类型及接口

[MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]

3.8. MI_HDMI_OutputMode_e

- 说明
定义 HDMI 输出模式类型
- 定义

```

typedef enum

{

    E_MI_HDMI_OUTPUT_MODE_HDMI = 0,

    E_MI_HDMI_OUTPUT_MODE_HDMI_HDCP,

    E_MI_HDMI_OUTPUT_MODE_DVI,

```

```
        E_MI_HDMI_OUTPUT_MODE_DVI_HDCP,

        E_MI_HDMI_OUTPUT_MODE_MAX,

    } MI_HDMI_OutputMode_e;
```

- 成员

表3-7

成员名称	描述
E_MI_HDMI_OUTPUT_MODE_HDMI	HDMI 输出模式
E_MI_HDMI_OUTPUT_MODE_HDMI_HDCP	HDMI + DHCP 输出模式
E_MI_HDMI_OUTPUT_MODE_DVI	DVI 输出模式
E_MI_HDMI_OUTPUT_MODE_DVI_HDCP	DVI + DHCP 输出模式

- 相关数据类型及接口

[MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]

3.9. MI_HDMI_ColorType_e

- 说明

定义 HDMI 颜色空间类型。

- 定义

```
typedef enum

{

    E_MI_HDMI_COLOR_TYPE_RGB444 = 0,

    E_MI_HDMI_COLOR_TYPE_YCBCR422,

    E_MI_HDMI_COLOR_TYPE_YCBCR444,
```

```
    E_MI_HDMI_COLOR_TYPE_YCBCR420,  
  
    E_MI_HDMI_COLOR_TYPE_MAX  
  
} MI_HDMI_ColorType_e;
```

- 成员

表3-8

成员名称	描述
E_MI_HDMI_COLOR_TYPE_RGB444	RGB444 输出模式
E_MI_HDMI_COLOR_TYPE_YCBCR422	YCBCR422 输出模式
E_MI_HDMI_COLOR_TYPE_YCBCR444	YCBCR444 输出模式
E_MI_HDMI_COLOR_TYPE_YCBCR420	YCBCR420 输出模式

- 相关数据类型及接口

[MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]

3.10. MI_HDMI_DeepColor_e

- 说明

定义 HDMI 深色模式类型。

- 定义

```
typedef enum  
{  
  
    E_MI_HDMI_DEEP_COLOR_24BIT = 0x00,  
  
    E_MI_HDMI_DEEP_COLOR_30BIT,  
  
    E_MI_HDMI_DEEP_COLOR_36BIT,
```

```
        E_MI_HDMI_DEEP_COLOR_48BIT,

        E_MI_HDMI_DEEP_COLOR_MAX,

    } MI_HDMI_DeepColor_e;
```

- 成员

表3-9

成员名称	描述
E_MI_HDMI_DEEP_COLOR_24BIT	HDMI Deep Color 24bit 模式
E_MI_HDMI_DEEP_COLOR_30BIT	HDMI Deep Color 30bit 模式
E_MI_HDMI_DEEP_COLOR_36BIT	HDMI Deep Color 36bit 模式
E_MI_HDMI_DEEP_COLOR_48BIT	HDMI Deep Color 48bit 模式

- 相关数据类型及接口

[MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]

3.11. MI_HDMI_SampleRate_e

- 说明

定义 HDMI 音频输出采样率类型。

- 定义

```
typedef enum

{

    E_MI_HDMI_AUDIO_SAMPLERATE_UNKNOWN = 0,

    E_MI_HDMI_AUDIO_SAMPLERATE_32K = 1,

    E_MI_HDMI_AUDIO_SAMPLERATE_44K = 2,
```

```

E_MI_HDMI_AUDIO_SAMPLERATE_48K = 3,

E_MI_HDMI_AUDIO_SAMPLERATE_88K = 4,

E_MI_HDMI_AUDIO_SAMPLERATE_96K = 5,

E_MI_HDMI_AUDIO_SAMPLERATE_176K = 6,

E_MI_HDMI_AUDIO_SAMPLERATE_192K = 7,

E_MI_HDMI_AUDIO_SAMPLERATE_MAX,

} MI_HDMI_SampleRate_e;

```

- 注意事项
 - 当AP在设置采样率的时候，需要匹配Audio模块的采样率，HDMI的采样率需和Audio模块的采样率保持一致。
- 相关数据类型及接口

[MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]

3.12. MI_HDMI_BitDepth_e

- 说明

定义 HDMI 音频输出采样位宽类型。
- 定义

```

typedef enum

{

E_MI_HDMI_BIT_DEPTH_UNKNOWN = 0,

E_MI_HDMI_BIT_DEPTH_8 = 8,

E_MI_HDMI_BIT_DEPTH_16 = 16,

E_MI_HDMI_BIT_DEPTH_18 = 18,

E_MI_HDMI_BIT_DEPTH_20 = 20,

```



```

E_MI_HDMI_BIT_DEPTH_24 = 24,

E_MI_HDMI_BIT_DEPTH_32 = 32,

E_MI_HDMI_BIT_DEPTH_BUTT

}MI_HDMI_BitDepth_e;

```

- 相关数据类型及接口

[MI_HDMI_SetAttr](#) [#26-mi_hdmi_setattr]v

3.13. MI_HDMI_SinkInfo_t

- 说明

定义 HDMI Sink 端能力级结构体。

- 定义

```

typedef struct MI_HDMI_Sink_Info_s
{

    MI_BOOL bConnected;

    MI_BOOL bSupportHdmi;

    MI_HDMI_TimingType_e eNativeTimingType;

    MI_BOOL abVideoFmtSupported[E_MI_HDMI_TIMING_MAX];

    MI_BOOL bSupportYCbCr444;

    MI_BOOL bSupportYCbCr422;

    MI_BOOL bSupportYCbCr;

    MI_BOOL bSupportxvYcc601;

    MI_BOOL bSupportxvYcc709;

    MI_U8 u8MdBit;

    MI_BOOL abAudioFmtSupported[MI_HDMI_MAX_ACAP_CNT];
}

```

```
MI_U32
au32AudioSampleRateSupported[MI_HDMI_MAX_AUDIO_SAMPLE_RATE_CNT];

MI_U32 u32MaxPcmChannels;

MI_U8 u8Speaker;

MI_U8 au8IdManufactureName[4];

MI_U32 u32IdProductCode;

MI_U32 u32IdSerialNumber;

MI_U32 u32WeekOfManufacture;

MI_U32 u32YearOfManufacture;

MI_U8 u8Version;

MI_U8 u8Revision;

MI_U8 u8EdidExternBlockNum;

MI_U8 au8IeeRegId[3]; //IEEE registration identifier

MI_U8 u8PhyAddr_A;

MI_U8 u8PhyAddr_B;

MI_U8 u8PhyAddr_C;

MI_U8 u8PhyAddr_D;

MI_BOOL bSupportDviDual;

MI_BOOL bSupportDeepColorYcbcr444;

MI_BOOL bSupportDeepColor30Bit;

MI_BOOL bSupportDeepColor36Bit;

MI_BOOL bSupportDeepColor48Bit;

MI_BOOL bSupportAi;

MI_U32 u8MaxTmdsClock;
```

```
MI_BOOL bILatencyFieldsPresent;

MI_BOOL bLatencyFieldsPresent;

MI_BOOL bHdmiVideoPresent;

MI_U8 u8VideoLatency;

MI_U8 u8AudioLatency;

MI_U8 u8InterlacedVideoLatency;

MI_U8 u8InterlacedAudioLatency;

} MI_HDMI_SinkInfo_t;
```

- 成员

表3-10

成员名称	描述
bConnected	设备是否连接
bSupportHdmi	设备是否支持 HDMI，如果不支持，则为 DVI 设备
eNativeVideoFormat	显示设备物理分辨率。
abVideoFmtSupported	视频能力集。 MI_TRUE：支持这种显示格式； MI_FALSE：不支持。
bSupportYCbCr	是否支持 YCBCR 显示。 MI_TRUE：支持 YCBCR 显示； MI_FALSE：只支持 RGB 显示。
bSupportxvYCC601	是否支持 xvYCC601 颜色格式
bSupportxvYCC709	是否支持 xvYCC709 颜色格式
u8MDBit	xvYCC601 支持的传输 profile:1:P0,2:P1,4:P2
bAudioFmtSupported 成员名称	音频能力集，请参考 EIA-CEA-861-D 表 37 描述 MI_TRUE：支持这种显示格式；

	MI_FALSE：不支持。
u32AudioSampleRateSupported	音频采样率能力集，0 为非法值，其他为支持的音频采样率。
u32MaxPcmChannels	音频最大的 PCM 通道数。
u8Speaker	扬声器位置，请参考 EIA-CEA-861-D 中 SpeakerDATABlock 的定义。
u8IDManufactureName	设备厂商标识。
u32IDProductCode	设备 ID
u32IDSerialNumber	设备序列号
u32WeekOfManufacture	设备生产日期(周)
u32YearOfManufacture	设备生产日期(年)
u8Version	设备版本号
u8Revision	设备子版本号
au8EDIDExternBlockNum	EDID 扩展块数目
au8IEERegId	24-bit IEEE Registration Identifier (0x000C03)。
u8PhyAddr_A	CEC 物理地址 A
u8PhyAddr_B	CEC 物理地址 B
u8PhyAddr_C	CEC 物理地址 C
u8PhyAddr_D	CEC 物理地址 D
bSupportDVIDual	是否支持 DVI dual-link 操作。
bSupportDeepColorYCBCR444	是否支持 YCBCR 4:4:4 Deep Color 模式。
bSupportDeepColor36Bit	是否支持 Deep Color 36bit模式
成员名称	描述

bSupportDeepColor48Bit	是否支持 Deep Color 48bit模式
bSupportAI	是否支持 Supports_AI 模式
u8MaxTMDSCLK	最大 TMDS 时钟
bLatencyFieldsPresent	延时标志位
bHDMIVideoPresent	Video_Latency 和 Audio_Latency fields 是否存在。
u8VideoLatency	视频延时
u8AudioLatency	音频延时
u8InterlacedVideoLatency	隔行视频模式下的视频延时
u8InterlacedAudioLatency	隔行视频模式下的音频延时

- 相关数据类型及接口

[MI_HDMI_GetSinkInfo](#) [#210-mi_hdmi_getsinkinfo]

3.14. MI_HDMI_Edid_t

- 说明

定义 HDMI EDID 信息结构体

- 定义

```
typedef struct MI_HDMI_Edid_s
{
    MI_BOOL bEdidValid;

    MI_U32 u32Edidlength;

    MI_U8 au8Edid[512];
} MI_HDMI_Edid_t;
```

- 成员

表3-11

成员名称	描述
bEdidValid	EDID 信息是否有效
u32Edidlength	EDID 信息数据长度,一般为256 Byte
au8Edid	au8Edid

- 相关数据类型及接口

[MI_HDMI_ForceGetEdid](#) [#212-mi_hdmi_forcegetedid]

3.15. MI_HDMI_InfoFrameType_e

- 说明

定义 HDMI 信息帧类型。

- 定义

```
typedef enum
{
    E_MI_INFOFRAME_TYPE_AVI = 0,
    E_MI_INFOFRAME_TYPE_SPD,
    E_MI_INFOFRAME_TYPE_AUDIO,
    E_MI_INFOFRAME_TYPE_MAX
} MI_HDMI_InfoFrameType_e;
```

- 成员

表3-12

成员名称	描述
E_MI_INFOFRAME_TYPE_AVI	AVI 信息帧类型
E_MI_INFOFRAME_TYPE_SPD	SPD 信息帧类型
E_MI_INFOFRAME_TYPE_AUDIO	AUDIO 信息帧类型

- 相关数据类型及接口

[MI_HDMI_DeInit](#) [#23-mi_hdmi_deinit]

[MI_HDMI_SetInfoFrame](#) [#215-mi_hdmi_setinfoframe]

3.16. MI_HDMI_AviInfoFrameVer_t

- 说明

定义 AVI 信息帧结构体。

- 定义

```
typedef struct MI_HDMI_AviInfoFrameVer_s
{
    MI_BOOL enableAFDoverWrite;

    MI_U8 A0Value;

    MI_HDMI_ColorType_e eColorType;

    MI_HDMI_Colorimetry_e eColorimetry;

    MI_HDMI_ExtColorimetry_e eExtColorimetry;

    MI_HDMI_YccQuantRange_e eYccQuantRange;

    MI_HDMI_TimingType_e eTimingType; //trans to
    MS_VIDEO_TIMING in impl

    MI_HDMI_VideoAfdRatio_e eAfdRatio;
```

```
MI_HDMI_ScanInfo_e eScanInfo;

MI_HDMI_AspectRatio_e eAspectRatio;

} MI_HDMI_AviInfoFrameVer_t;
```

- 成员

表3-13

成员名称	描述
eColorType	颜色空间类型
eScanInfo	扫描信息枚举变量
eAspectRatio	幅形比枚举变量
eExtColorimetry	扩展色彩空间矩阵
eTimingType	视频格式类型
eYccQuantRange	YCC 色彩空间范围

- 注意事项
 - 各成员变量的枚举成员详见 `mi_hdmi_datatype.h` 文件。
- 相关数据类型及接口

[MI_HDMI_SetInfoFrame](#) [#215-mi_hdmi_setinfoframe]

[MI_HDMI_GetInfoFrame](#) [#216-mi_hdmi_getinfoframe]

3.17. MI_HDMI_AudInfoFrameVer_t

- 说明

定义 AUDIO 信息帧结构体。
- 定义


```
typedef struct MI_HDMI_AudInfoFrameVer_s
{
    MI_U32 u32ChannelCount;

    MI_HDMI_AudioCodeType_e eAudioCodeType;

    MI_HDMI_SampleRate_e eSampleRate;
} MI_HDMI_AudInfoFrameVer_t;
```

• 成员

表3-14

成员名称	描述
u32ChannelCount	通道个数。
eAudioCodeType	音频数据编码类型。
eSampleRate	采样频率。 推荐设 0 0：表由码流头决定。 1：32kHz 2：44.1 kHz 3：48 kHz 4：88.2 kHz 5：96 kHz 6：176.4 kHz 7：192 kHz

• 相关数据类型及接口

[MI_HDMI_SetInfoFrame](#) [#215-mi_hdmi_setinfoframe]

[MI_HDMI_GetInfoFrame](#) [#216-mi_hdmi_getinfoframe]

3.18. MI_HDMI_SpdInfoFrame_t

- 说明

定义 SPD 信息帧结构体。

- 定义

```
typedef struct MI_HDMI_SpdInfoFrame_s
{
    MI_U8 au8VendorName[8];

    MI_U8 au8ProductDescription[16];
} MI_HDMI_SpdInfoFrame_t;
```

- 成员

表3-15

成员名称	描述
au8VendorName[8]	卖方名称
au8ProductDescription[16]	产品描述符

- 相关数据类型及接口

[MI_HDMI_SetInfoFrame](#) [#215-mi_hdmi_setinfoframe]

[MI_HDMI_GetInfoFrame](#) [#216-mi_hdmi_getinfoframe]

3.19. MI_HDMI_InfoFrame_Unit_u

- 说明

定义 HDMI 信息帧联合体。

- 定义

```
typedef union
{
```

```
MI_HDMI_Avi InfoFrameVer_t stAviInfoFrame;

MI_HDMI_AudInfoFrameVer_t stAudInfoFrame;

MI_HDMI_SpdInfoFrame_t stSpdInfoFrame;

} MI_HDMI_InfoFrameUnit_u;
```

- 成员

表3-16

成员名称	描述
stAviInfoFrame	AVI 信息帧
stAudInfoFrame	AUD 信息帧
stSpdInfoFrame	SPD 信息帧

- 相关数据类型及接口

[MI_HDMI_SetInfoFrame](#) [#215-mi_hdmi_setinfoframe]

[MI_HDMI_GetInfoFrame](#) [#216-mi_hdmi_getinfoframe]

3.20. MI_HDMI_AnalogDrvCurrent_t

- 说明

定义 HDMI 通道驱动电流结构体。

- 定义

```
typedef struct MI_HDMI_AnalogDrvCurrent_s
{

    MI_U8 u8DrvCurTap1Ch0;

    MI_U8 u8DrvCurTap1Ch1;
```

```
MI_U8 u8DrvCurTap1Ch2;

MI_U8 u8DrvCurTap1Ch3;

MI_U8 u8DrvCurTap2Ch0;

MI_U8 u8DrvCurTap2Ch1;

MI_U8 u8DrvCurTap2Ch2;

MI_U8 u8DrvCurTap2Ch3;

} MI_HDMI_AnalogDrvCurrent_t;
```

• 成员

表3-17

成员名称	描述
u8DrvCurTap1Ch0	Tap1 通道0驱动电流配置
u8DrvCurTap1Ch1	Tap1 通道1驱动电流配置
u8DrvCurTap1Ch2	Tap1 通道2驱动电流配置
u8DrvCurTap1Ch3	Tap1 通道3驱动电流配置
u8DrvCurTap2Ch0	Tap2 通道0驱动电流配置
u8DrvCurTap2Ch1	Tap2 通道1驱动电流配置
u8DrvCurTap2Ch2	Tap2 通道2驱动电流配置
u8DrvCurTap2Ch3	Tap2 通道3驱动电流配置

• 相关数据类型及接口

[MI_HDMI_SetAnalogDrvCurrent](#) [#217-mi_hdmi_setanalogdrvcurrent]

4. 错误码

表4-1 HDMI API 错误码

错误代码	宏定义	描述
0xA00B2011	MI_ERR_HDMI_INVALID_PARAM	非法参数
0xA00B2001	MI_ERR_HDMI_INVALID_DEVID	无效设备ID
0xA00B2018	MI_ERR_HDMI_DRV_FAILED	HDMI驱动返回失败
0xA00B2015	MI_ERR_HDMI_NOT_INIT	HDMI 未初始化
0xA00B2008	MI_ERR_HDMI_NOT_SUPPORT	不支持操作
0xA00B21E0	MI_ERR_HDMI_UNSupport_TIMING	不支持输出timing
0xA00B21E1	MI_ERR_HDMI_UNSupport_COLORTYPE	不支持颜色类型
0xA00B21E2	MI_ERR_HDMI_UNSupport_ACODETYPE	不支持的音频编码类型
0xA00B21E3	MI_ERR_HDMI_UNSupport_AFREQ	不支持的音频频率
0xA00B21E4	MI_ERR_HDMI_EDID_HEADER_ERR	EDID头信息解析失败
0xA00B21E5	MI_ERR_HDMI_EDID_DATA_ERR	EDID数据错误

5. PROCFS介绍

5.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_hdmi/mi_hdmi0

----- HDMI0 Dev Info -----
-----
InitFlag   AvMute   PowerOn
   Y       N       Y
EnableVideo TimingType OutputMode ColorType DeepColorMode
   1         9         0         2         4
EnableAudio IsMultiChannel BitDepth  CodeType  SampleRate
   1           0         16         0         3
```

• 调试信息分析

记录当前HDMI的使用状况以及device属性可以动态地获取到这些信息，方便调试和测试。

• 参数说明

</table

参数		描述
device info	InitFlag	HDMI模块是否初始化 Y：已初始化 N：未初始化
	AvMute	音视频是否处于MUTE状态 Y：被MUTE N：未被MUTE
	PowerOn	HDMI是否使能电源管理 Y：是 N：否
	EnableVideo	是否使能视频 0：未使能 1：使能
	TimingType	当前HDMI的输出分辨率 取值范围[0~ E_MI_HDMI_TIMING_MAX]
	OutputMode	HDMI输出模式 0：HDMI模式 2：DVI模式 1 / 3为HDCP模式，当前暂不支持

ColorType	颜色空间 0 : RGB44 1 : YUV422 2 : YUV444 3 : YUV420
DeepColorMode	色彩位深 0 : 8Bit 1 : 10Bit 2 : 12Bit 3 : 16Bit
EnableAudio	是否使能音频 0 : 未使能 1 : 使能
IsMultiChannel	是否多通道 0 : 否 1 : 是
BitDepth	Audio sampling bit depth 8: 8Bit 16: 16Bit 32: 32Bit
CodeType	Compression format for audio output 0: PCM 1: Non-PCM
SampleRate	Audio output sample rate 0: unknown 1: 32K 2: 44K 3: 48K 4: 88K 5: 96K 6: 176K 7: 192K